

ARENA: Asynchronous Reconfigurable Accelerator Ring to Enable Data-Centric Parallel Computing

Cheng Tan, Chenhao Xie, Tong Geng, Andres Marquez, Antonino Tumeo, Kevin Barker, and Ang Li

Abstract—The next generation HPC and data centers are likely to be reconfigurable and data-centric due to the trend of hardware specialization and the emergence of data-driven applications. In this paper, we propose ARENA – an asynchronous reconfigurable accelerator ring architecture as a potential scenario on how the future HPC and data centers will be like. Despite using the coarse-grained reconfigurable arrays (CGRAs) as the substrate platform, our key contribution is not only the CGRA-cluster design itself, but also the ensemble of a new architecture and programming model that enables asynchronous tasking across a cluster of reconfigurable nodes, so as to bring specialized computation to the data rather than the reverse. We presume distributed data storage without asserting any prior knowledge on the data distribution. Hardware specialization occurs at runtime when a task finds the majority of data it requires are available at the present node. In other words, we dynamically generate specialized CGRA accelerators where the data reside. The asynchronous tasking for bringing computation to data is achieved by circulating the task token, which describes the dataflow graphs to be executed for a task, among the CGRA cluster connected by a fast ring network. Evaluations on a set of HPC and data-driven applications across different domains show that ARENA can provide better parallel scalability with reduced data movement (53.9%). Compared with contemporary compute-centric parallel models, ARENA can bring on average $4.37\times$ speedup. The synthesized CGRAs and their task-dispatchers only occupy 2.93mm^2 chip area under 45nm process technology and can run at 800MHz with on average 759.8mW power consumption. ARENA also supports the concurrent execution of multi-applications, offering ideal architectural support for future high-performance parallel computing and data analytics systems.

Index Terms—Compute-Flow-Architecture, Runtime Reconfiguration, Asynchronous Parallel Execution, Abstract Machine Model.



1 INTRODUCTION

With the slowing down of Moore’s Law [53], future computer systems will need to resort to domain-specific accelerators for continuous performance scaling [27], [67] under the same power envelope. This is especially the case for HPC and data centers, as we are quickly entering an era of extreme heterogeneity [76], characterized by cluster nodes integrating a multitude of cooperating accelerators [35], [41], [46], [65].

While integrating domain-specific accelerators (DSAs) into HPC and data centers provides considerable efficiency gains [35], [41], [59], [68], it leads to enormous complexities as well. First, DSAs can only be economically designed for ubiquitous computational patterns in applications, while the workloads currently running in HPC and data centers are converging towards mixed workflows that include scientific simulation, machine learning, data analytics, etc. Second, managing various (typically loosely coupled) accelerators across many nodes significantly complicates programming models and the software infrastructure. In both HPC and data centers, the accelerators typically need to be shared among multiple users, often using multiple nodes for applications with divergent characteristics. Since we could not design hardware accelerators for all seen and unseen kernels, reconfigurable architecture, which allows specialization after system deployment and even during system execution,

promises to be a wise solution. For example, a user might launch a graph convolutional neural network inference while another one starts a breadth-first search on the same input graph that is allocated across all the machine nodes. By enabling dynamic reconfiguration, different kernels in one application and even different applications can be efficiently accelerated simultaneously in the HPC system during runtime while Field Programmable Gate Arrays (FPGAs) have already been deployed at large scale in some data centers [65], they may suffer from limited frequency and energy-efficiency compared with ASICs, as well as long reconfiguration time (e.g. in milliseconds) due to bit-level reconfigurability. Coarse-grained reconfigurable arrays (CGRAs), which integrate highly optimized functional units (rather than fundamental lookup tables or LUTs) and offering reconfigurability at the word-level, emerge as a promising alternative choice [64]. The rapid reconfiguration [24] makes dynamic formalization of hardware accelerators at runtime becomes feasible, and even plausible.

Conventional large-scale HPC clusters implicitly assuming homogeneous node-configuration and bulk-synchronous-parallel (BSP) execution model suffer from three low-utilization challenges regarding per-node data locality: (1) unbalanced data distribution among homogeneous nodes may lead to unbalanced workload and poor utilization; (2) If data is not locally available, during the long-time remote data fetching, the compute units can be idle; (2) Even worse, these idle units or idle nodes cannot be reclaimed for other tasks despite the node may hold their desired

- The authors are with Pacific Northwest National Laboratory, Richland, WA, 99354.
E-mail: cheng.tan@pnnl.gov

data. As emerging workloads become more dynamic and data-driven, decentralized asynchronous task management is highly desired, while data locality becomes a crucial factor for the system design [18], [27], [67]. This is largely due to the observation that the energy cost of data-movement significantly overweights the energy cost of computing them [36], [43], [52], which is particularly the case when migrating data through the interconnect network (e.g., the power budget is ~ 5.5 watts per full bi-directional NVLink port [1]). While software solutions such as *active messages* [16], [17] and remote procedure call (RPC) mitigate the problem by pushing computation to the nodes where data resides, they suffer from considerable overhead due to the lack of architectural support. Existing applications however still widely adopt the BSP model [14], [74] alternating phases of (parallel) local computation with phases of global communication. Note that the BSP model implicitly assumes that the majority of the time can be spent in easily parallelizable computation phases, with limited data movement. However, the exponential growth in the availability of data and the emergence of new applications, radically changed the balance.

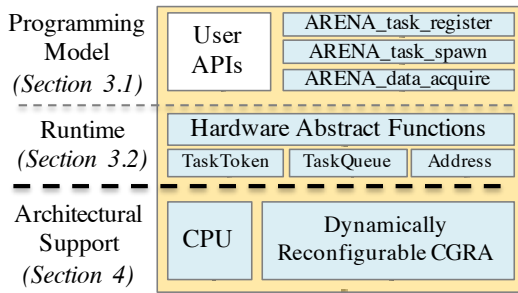


Fig. 1: Overall ARENA design stack.

Bridging coarse-grained reconfigurable architecture with locality-oriented asynchronous parallel task execution, in this paper, we propose ARENA — an Asynchronous REcoNfigurabe Accelerator ring architecture & runtime, to enable data-centric computation flow paradigm aiming at significantly reducing unnecessary inter-node data movement. ARENA comprises multiple CGRAs interconnected in a ring to provide quick reconfiguration at runtime. The tasks of the application, tagged as task tokens, are injected into the ring when dependencies are satisfied. Task-specialized CGRA accelerators can be constructed at runtime using the specification embedded in the task token. When task tokens are streaming around the ring, each node can verify whether a task should be executed locally (based on data locality and resource availability). In other words, ARENA is dynamically sharing hardware resource according to data locality. This is in contrast with conventional systems sharing time-slots according to hardware availability. In summary, this paper thus makes the following contributions:

- **Architecture:** we proposed a multi-CGRA cluster architecture (Section 4) that can be dynamically reconfigured at runtime, enabling asynchronous parallel execution for multi-users with very little overhead.
- **Runtime:** we proposed a flexible runtime (Section 3.2) enabling task-token delivery along the ring network based on data locality. Compared to compute-centric execution models, the proposed runtime can significantly mitigate data movement and improve performance.

- **Programming Model:** we proposed a data-centric programming model (Section 3.1) with easy-to-use APIs to facilitate the programming for ARENA architecture and runtime. An LLVM-based compiler toolchain is constructed to support the programming model.
- **Evaluation:** RTL simulation results using practical HPC and data analytics applications (Section 5) show that ARENA can provide better parallel scalability with reduced data movement (53.9%) and bring $2.17\times$ and $4.37\times$ speedup over traditional compute-centric approach with and without CGRA acceleration. The CGRA-based ARENA prototype only costs $2.93mm^2$ chip area in 45nm and can work at 800MHz with 759.8mW power consumption per node, showing significant advantages over the present architectural design in the current HPC.

2 MOTIVATION

Scientific simulation, where linear solvers iterate on data organized in dense (structured sparse) matrices or tensors, typically easy to divide in equally sized tiles, represents the premier HPC workload [5], [6], [60], [73], [78]. However, emerging HPC applications, targeting areas such as power grid dynamics [4], seismic risk assessment [51], urban systems simulation, and microbiome analysis [3], will likely combine together traditional scientific simulation with advanced data analytics and machine learning. The datasets for these applications are much less structured, and thus more difficult to organize in **regular** and **partitionable** data structures. Applications will alternate phases of scientific simulations with **regular** behaviors, to phases where the computation happens on sparse data structures (e.g., sparse matrices, graph traversal) that induce unpredictable fine-grained data accesses and **irregular** behaviors. This scenario provides a clear opportunity for adaptability to diverse behaviors with reconfigurable hardware, while at the same time makes the current HPC programming models inadequate. In the following, we describe three major existing multi-node computing paradigms to motivate the ARENA design.

2.1 Baseline-1: Compute-Centric BSP Execution Model

HPC systems typically rely on the classical Bulk Synchronous Parallel (BSP) programming model in which a process is assigned to a processor or an entire node, and communication typically happens through message passing with libraries such as MPI. In the BSP model, the computation proceeds as a series of global supersteps: concurrent computation, where every participating nodes perform parallel computations on local data; communication, where nodes exchange data among them (with various, algorithm dependent, patterns); and barrier synchronization, to align execution of nodes.

The BSP model assumes that data are partitioned and distributed across nodes and rarely moves, to facilitate the local computation super-steps. Otherwise, the message-passing based communication super-steps would dominant the execution time. While this model works well for applications with easily partitionable data, regular computation, and limited, structured communication, it starts to experience significant limitations when workloads exhibit irregular behaviors (skewed data distributions, high synchronization intensity, irregular communication patterns). For these reasons, we consider the BSP model as *Compute-Centric*.

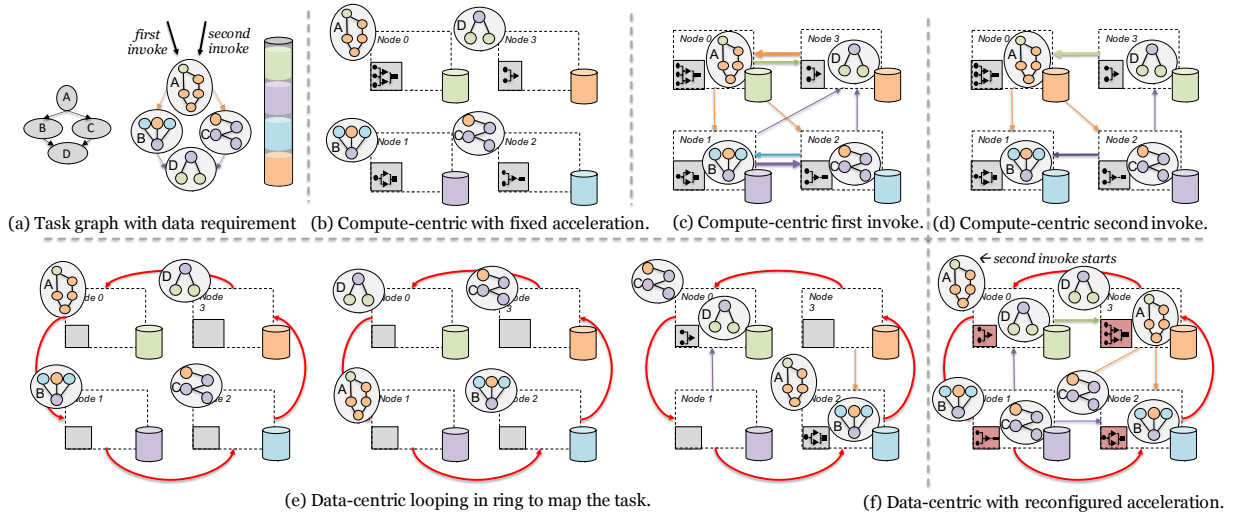


Fig. 2: Motivating example of the traditional compute-centric and ARENA data-centric execution models.

Consider an application with the (hierarchical) task graph in Figure 2(a), where the computation is split into 4 high-level tasks, each one executing task-partitioned computational kernels where the subtasks require data from other nodes. When employing a BSP model, both the data allocation and the distribution of the high-level tasks are fixed for the entire application execution. Hence, if a subtask needs data available in another node, it needs to initiate a communication phase, load the remote data, and synchronize to avoid hazards. Despite the latest high-performance designs can exploit mechanisms such as *remote-direct-memory-access* (one-sided communication, not requiring a blocking receive with implicit synchronization from the remote node), prefetching and data migration, when the remote accessing are frequent, the bandwidth gap between local memory and remote memory (which needs to be accessed via the network) still remains the major performance (and consequently energy) concern. For example, if, as typical in HPC applications, tasks are executed in a loop, and they contend on the same data blocks (e.g., Tasks A, B, and C in Figure 2), data migration may trigger even more data movement and synchronization, as the actual data distribution and access patterns are unknown before runtime.

Limitations – While compute-centric BSP long remains the standard model in HPC, its adoption in emerging HPC applications may be limited by data movement and synchronization. HPC practitioners have introduced asynchronous multi-task runtimes to tackle the limitations of the BSP model. Besides migrating data blocks where the computation occurs, these runtimes often allow tasks to migrate where the data reside, via approaches such as active messages and remote procedure calls, following the *data-centric* models.

2.2 Baseline-2: Reconfigurable accelerators in HPC

Reconfigurable accelerators have been deployed at a massive scale in data centers to provide application-specific acceleration with improved power-/area-efficiency [32], [64], [65]. Some institutions have also started hosting clusters with FPGA to perform research in HPC [2].

However, accelerators in HPC installations still generally adopt the BSP model, where a part of the local computation is offloaded to the accelerator itself. This also requires gathering desired data by the running tasks offloaded for execution. While FPGAs potentially allow acceleration of

workloads more diverse than conventional accelerators such as GPUs, their current usage in HPC application often entails static configuration for accelerating a small amount of kernels (e.g., FFT, GEMM). This approach matches with an offload accelerator model - tasks and computational kernels do not move in the system. On the other hand, a reconfigurable architecture potentially allows to dynamically adjust the configuration at runtime, being able to accelerate divergent tasks with the computation proceeds. Despite the potential, the excessive reconfiguration overheads (typically in milliseconds) makes such an approach very costly.

Limitations – Reconfigurable accelerators make it possible to accelerate a more diverse workload, but current practice in heterogeneous HPC still leverages the offload model. The lack of low-latency runtime reconfiguration also limits the chances of large-scale task migration across the whole system. On the other hand, the data-centric execution model allows different tasks to work concurrently, despite using the same set of data in a node. CGRAs, as an alternative reconfigurable solution [64] to FPGAs [24], offer significantly reduced reconfiguration time with coarser-grained reconfiguration, making flexible runtime architecture adjustment becomes possible.

2.3 Baseline-3: Data-Centric Software Approaches

Many programming models have been leveraged or designed to allow data-centric execution on multi-node systems. SSMP [62] can operate on shared memory machines and support dynamic detection of dependencies between tasks. The implicitly shared memory management and dependence detection improve the programmability at the cost of increased synchronization overhead. *Remote procedure call* (RPC) achieves near-data-computation based on prior knowledge about the exact distribution of data. X10 [21] and Chapel [20] allow the programmers to control where to place the data and tasks. Similarly, Legion [12] enables explicit, programmer-controlled movement of data and placement of asynchronously spawned tasks, based on locality information. Legion employs a Cilk-like [15] algorithm for locality-aware task stealing.

Towards data-centric programming, MapReduce [25] programmers think in a data-centric fashion: they focus more on handling the sets of data records, rather than managing fine-grained threads, processes, communication,

and coordination [9]. However, MapReduce constrains its usage to batch-processing tasks, which falls in the BSP scope. To accommodate the emerging data-driven applications with irregular and unpredictable data access patterns, data-centric execution with asynchronous task-spawn should be enabled with hardware support. Meanwhile, the synchronization and task dependencies should be specified by the programmers to eliminate the unnecessary performance and energy overhead rather than forced by the programming model (e.g., remote procedure need to return to local in RPC, all the spawned tasks of the same ancestor need to join eventually in Legion). Besides the data locality, the runtime should also consider the computing resource utilization when reconfigurable accelerators are deployed and shared by multi-users in HPC/data-center environments.

Limitations – The high-level software framework and runtime facilitate the asynchronous execution of tasks and attempt to take advantage of data locality. Unfortunately, existing frameworks based on software solutions incur considerable overhead. The lack of hardware reconfiguration also limits the benefit from application-specific design and heterogeneity.

2.4 Compute-Flow ARENA Execution Model

We propose ARENA to address the limitation of the three aforementioned baseline. ARENA includes a novel programming model targeting asynchronous data-centric execution paradigm. As shown in Figure 2, all the configurable nodes in ARENA are connected by a ring network to bring the specialized computation to the data rather than the reverse to minimize data movement. Each reconfigurable node mainly contains a CGRA (detailed in Section 4.3) that supports real-time reconfiguration and simultaneous execution of multi-tasks.

3 ARENA PROGRAMMING MODEL

Being the interface between software and hardware, the ARENA programming model defines a list of API functions in Table 1. On one hand, in order to program an ARENA abstract machine, a software programmer has to define their user-logic as task functions, and rely on the *User APIs* to operate the abstract machine. Please note that although in this work we use CGRAs as the hardware testbed, it is only one of the the possible instantiations of the ARENA abstract machine model (AMM). On the other hand, in order to support ARENA software and run ARENA program, an alternative architecture has to support the *Hardware Abstract Functions* here.

3.1 ARENA Programming Interface

To program an ARENA abstract machine, the programmers first partition their application into tasks and register the defined tasks to the ARENA runtime. Ideally, the partition can separate the working set into a bunch of continuous data segments, where each task accounts for a segment. ARENA does not limit the granularity of a task, which can be extremely fine-grained or coarse-grained. While ARENA works perfectly when the data is locally available for a task, we understand this is not always feasible. When remote data access is inevitable, the application can either spawn a new task for the remote data, or explicitly initiate the data-movement through the *data-transfer-network*.

Function & Construct	Description
User Defined Function	
void my_task (Address TASKstart, Address TASKend, float PARAM)	A user can implement multiple different tasks to compose a single or multiple applications.
User APIs	
void ARENA_task_register (int TASKid, Address &my_kernel, bool isRoot)	Registers a kernel (e.g. <i>my_kernel</i>) with TASKid. The root task is launched by a CPU or a microcontroller once the system starts to run.
void ARENA_task_spawn (int TASKid, Address TASKstart, Address TASKend, float PARAM, Address REMOTEstart, Address REMOTEend)	Dynamically spawns a new TaskToken (<i>FROMnode</i> is automatically applied) that will be issued to the CoalesceUnit . The fields of a task token is explained in detail in Section 4.1.
Hardware Abstract Functions	
void ARENA_init (Address* local_start, Address* local_end)	Initializes <i>local_start</i> and <i>local_end</i> based on local data information.
TaskToken ARENA_arrive ()	Receives an incoming task from a remote node.
void ARENA_filter (TaskToken token, Address* local_start, Address* local_end, TaskQueue* SendQueue, TaskQueue* WaitQueue)	Detaches, Splits or passes tasks based on the <i>token</i> 's required data addresses (by comparing them with <i>local_start</i> and <i>local_end</i>).
bool ARENA_ready (TaskToken token)	Checks if the computing resources are available for executing the task <i>token</i> .
void ARENA_launch (TaskToken token, TaskQueue* CoalesceUnit)	Issues a task denoted by <i>token</i> either to a CPU or to a reconfigurable accelerator (e.g., CGRA). The spawned new tasks will be pushed into <i>CoalesceUnit</i> .
void ARENA_data_acquire (TaskToken token)	Acquires additional data from remote node via NIC.
TaskToken ARENA_coalesce (TaskQueue* CoalesceUnit)	Coalesces spawned tasks with continuous data range, identical required remote data range, and identical PARAM.
Base Constructs	
TaskToken	encapsulates a task.
Address	Local address of data.
TaskQueue	Buffers for task tokens.

TABLE 1: ARENA programming and hardware APIs.

```

1 // Users can define multiple different kernels,
2 // each with a specific task token ID.
3 #define BFS_TOKEN 1
4 int** local_M;
5 ...
6 void BFS_kernel(int TASKstart, int TASKend, int PARAM){
7   int lev = PARAM;
8   for(int i=TASKstart; i<TASKend; ++i) {
9     for(int j=0; j<SIZE; ++j) {
10      if(local_M[i][j] > lev) {
11        local_M[i][j] = lev;
12        ARENA_task_spawn(BFS_TOKEN, j, j+1, lev+1); }}}
13 void run() {
14   // Register kernels.
15   ARENA_task_register(BFS_TOKEN, &BFS_kernel, true);
16   // Launch ARENA runtime.
17   ARENA_runtime(); }
```

Fig. 3: Example of programming SSSP in ARENA.

Figure 3 shows an example on how to solve the single-source shortest path (SSSP) problem using a breadth-first search (BFS) kernel. The design traverses associated vertices until the shortest path(s) from a source node to all the other nodes are found. Without losing generality, we assume the graph, represented as an adjacency matrix, is distributively stored on all nodes and each node holds *SIZE/NODES* vertices (rows) of the entire graph (the adjacency matrix is in *SIZE × SIZE*, an initial value of ∞ indicates a connected edge while 0 implies no connection).

ARENA enables asynchronous data-centric execution by dynamically spawning new task-tokens among the nodes. Currently, all tasks need to be registered at the beginning. During runtime, task-tokens are circulating among all nodes in the ring. In case a node confirms it has the data required by a task-token (indicated by the starting & ending addresses $TASK_{start}$ and $TASK_{end}$) as well as sufficient hardware resources for runtime hardware specialization implied by the task-token, it takes out the token from the task-token-stream and executes it. A task will be executed by the CPU if no hardware specialization is provided. New tasks for remote nodes can be generated or spawned at any node. We currently rely on the programmer to determine the granularity of a spawned task (in other words, the data-range a spawned task designated). Fine-grained tasks facilitate asynchronous execution but increase scheduling overhead in the runtime. We discuss this tradeoff in detail in Section 3.2.

This is in contrast with the conventional compute-centric approach demanding frequent data communication and synchronization [19]. As each node maintains the vertex status locally, and no prior knowledge about vertex distribution is asserted, repeated all-to-all communications are essentially desired for broadcasting vertex updating information to associated nodes on the present frontier. Figure 9 shows the performance gain of ARENA for the SSSP application.

3.2 ARENA Runtime Support

Figure 4 and 5 illustrate the workflow of ARENA runtime executed per-node and the pseudo-code of the workflow, respectively. As can be seen, multiple tasks (marked with different colors) can be asynchronously executed in parallel. Note, the runtime can be supported by CPUs, GPUs, DSPs, or any other fixed or reconfigurable hardware substrate given the substrate realize the *Hardware Abstract Functions* and support the *Base Constructs*.

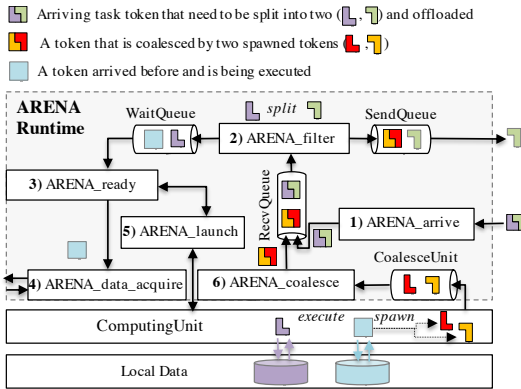


Fig. 4: ARENA runtime workflow.

We describe the runtime process below: Primarily, the task queues (line 3-4) and local data range (line 7) are initialized. We then proceed with 6 steps: **Step-(1):** All the incoming task tokens from the proceeding node will be appended to the RecvQueue (line 8-11). **Step-(2):** A token popped from the RecvQueue will be processed in the Filter, where a task can be split into multiple tasks, which are either buffered in the WaitQueue for local execution, or forwarded to the SendQueue, wait for being conveyed to the next node (line 21-23). The logic is: (I) if the task data range is irrelevant to the node's local data range, it is forwarded to SendQueue as it is; (II) if the task data range is a subset

```
1 void ARENA_runtime() {
2     bool terminate = false;
3     TaskQueue WaitQueue, RecvQueue;
4     TaskQueue SendQueue, CoalesceUnit;
5     TaskToken token;
6     Address local_start, local_end;
7     ARENA_init(&local_start, &local_end);
8     while(true) {
9         // Enqueue an arriving task token.
10        token = ARENA_arrive();
11        RecvQueue.enqueue(token);
12        // Check termination for all tasks.
13        token = RecvQueue.dequeue();
14        if(token.TASKid == TERMINATE and
15           WaitQueue.empty()) {
16            SendQueue.enqueue(token);
17            if(terminate) break;
18            terminate = true;
19            continue; }
20        terminate = false;
21        // Offload, split, or convey a task.
22        ARENA_filter(token, local_start, local_end,
23                    &SendQueue, &WaitQueue);
24        // Check availability for task execution.
25        token = WaitQueue.peek();
26        if(ARENA_ready(token)) {
27            WaitQueue.dequeue();
28            // Acquire data if necessary.
29            if(token.REMOTEend > token.REMOTEstart) {
30                ARENA_data_acquire(token);}
31            // Issue a task for execution and return
32            // without waiting for completion.
33            ARENA_launch(token, CoalesceUnit);{
34            // Task coalescing if possible.
35            token = ARENA_coalesce(CoalesceUnit);{
36            if(token)
37                RecvQueue.enqueue(token); }}
```

Fig. 5: ARENA runtime pseudocode.

of the node's local data range ($local_{start} \leq TASK_{start} \leq TASK_{end} \leq local_{end}$), implying the local availability of all the needed data for the task, the token will be pushed into WaitQueue for future execution; (III) if the data range indicated by the task is a superset of the node's local data range ($TASK_{start} \leq local_{start} \leq local_{end} \leq TASK_{end}$), it suggests that the task might be too coarse-grained. We split the task into three portions and spawn three new tasks. The one with data range $local_{start}$ to $local_{end}$ will be buffered in WaitQueue for local processing; the other two tasks are redirected to SendQueue; (IV) finally, if the task data range is partially aligned with the node's local data range, two new tasks will be spawned. The aligned part is buffered in WaitQueue while the mismatch part is forwarded to SendQueue. **Step-(3):** The runtime checks whether there is available resources for the token at the top of the WaitQueue to be executed (line 26). **Step-(4):** If so, the runtime verifies whether the task needs to incur any inevitable remote data access. If yes, the data will be acquired from the objective remote nodes (line 30) through the Data-Transfer-Network. **Step-(5):** when all demanding data are available, the task is issued to the computing resources for execution (line 33). **Step-(6)** As new tasks can be generated locally in a node during the task execution, to avoid too many tasks flooding the system, a CoalesceUnit (line 35) is designed to aggregate the newly generated tasks if the boundaries of their data-ranges coincide each other, and whether alternative key parameters (e.g., task-token carried partial-reduction variables) are the same. This can avoid the scenario that too many generated fine-grained tasks saturate the task-token network and the associated buffers. Finally, the runtime on a particular node terminates when

the TERMINATE token has been continuously received, and there is no pending tasks in the local WaitQueue (line 12-20).

4 ARENA CGRA-CLUSTER ARCHITECTURE

Our ARENA prototype in this work is built upon a CGRA cluster interconnected by a fast ring network. Figure 6(a) shows the overall design — multiple reconfigurable nodes are connected in a ring topology. Each node incorporates a micro-controller (e.g., a simplified CPU), a task dispatcher, a CGRA, a network interface (NIC), a DMA unit, and local memory storage. At runtime, task tokens are circulating along the ring. The dispatcher can split, offload, and forward a task token based on the data range as already discussed. We adopt the ring network topology to simplify the routing strategy and provide sufficient bandwidth for delivering the small-sized task tokens (~21 bytes) with up to 16 nodes evaluated in this paper. The ring network can also provide near optimal bandwidth for most collective communication [57], and can be easily built upon various physical network topology. We leave the exploration of alternative multi-node topology as a future work.

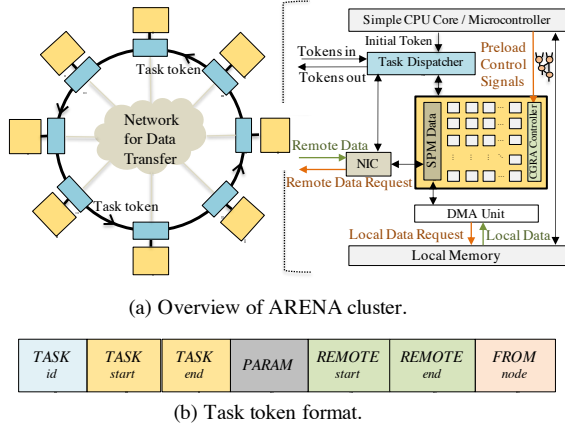


Fig. 6: ARENA cluster overview and task token format.

4.1 Task Token

A task is represented by a task token in ARENA, which can be dynamically spawned, executed, delivered, and split. Figure 6(b) shows the general format of the task token which comprises 7 fields: $TASK_{id}$ indicates the task will be executed, which is registered by the user (using `ARENA_task_register()`) before launching the ARENA runtime. During execution, the reconfigurable node will be dynamically configured (see Section 4.3) based on $TASK_{id}$. $TASK_{start}$ and $TASK_{end}$ together describe the data range for the task. $PARAM$ refers to a token-carried return value that is typically initialized by its parent task. This field is useful when performing collective operations (e.g., *reduction* and *accumulation*). For unavoidable remote data access, we use $REMOTE_{start}$ and $REMOTE_{end}$ to indicate the starting and ending addresses. $FROM_{node}$ labels the node where its parent task locates. Each task token thus requires 21 bytes in our prototype architecture (i.e., 4-bit each for $TASK_{id}$ and $FROM_{node}$; 4-byte each for the other fields).

4.2 Task Dispatcher

ARENA's task dispatcher mainly includes a task Filter Logic and three queues as shown in Figure 7. The Filter

Logic can offload, split, and convey a task token based on the data requirement (see Section 3.2). The NIC handles remote data requests from the task tokens in the WaitQueue. The WaitQueue will be acknowledged when the required remote data arrives at the data memory. The CGRA controller will then pop the acknowledged task token from the head of WaitQueue and reconfigure the CGRAs accordingly.

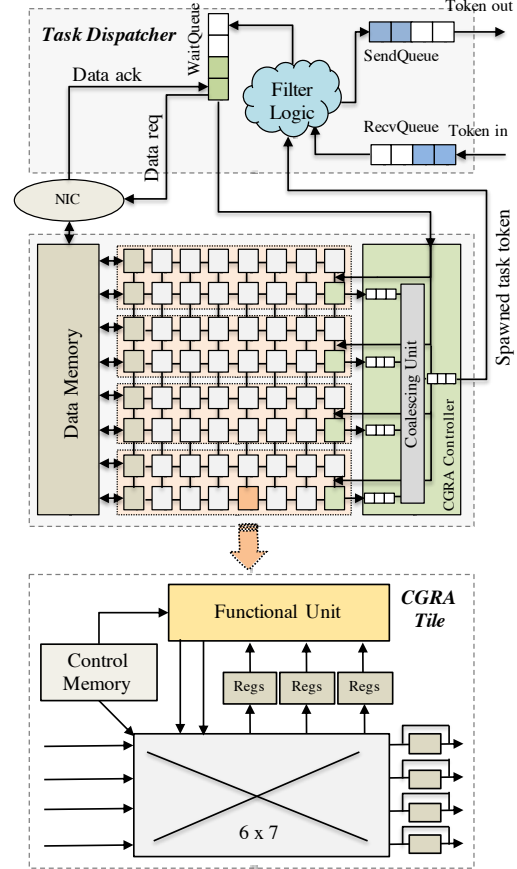


Fig. 7: Architecture of an ARENA node.

4.3 Reconfigurable CGRA Nodes and Toolchain

To achieve rapid dynamically reconfiguration, an ARENA node is prototyped with CGRA. The on-chip configuration memory is used for conserving the control signals for each task. The intra-node CGRA consists of 64 tiles connected in a mesh network. A scratchpad data memory conserves the data required for the computation. Note that both the control signals and the data are pre-loaded by the CPU/microcontroller through the DMA unit before launching the ARENA runtime. The CGRA communicates with the Task Dispatcher through the CGRA controller. The controller can offload tasks and coalesce spawned tokens through the Coalescing Unit.

CGRA Tile – As shown in Figure 7, each CGRA tile contains a functional unit, a scratchpad control memory, a crossbar switch, and three sets of registers. The functional unit supports all the basic operations (e.g., *add*, *mul*, *shift*, *select*, *branch*, *load*, *store*, etc). Control-divergence (i.e., the existence of multiple control flow paths) inside the loop kernel is supported through partial predication [34]. The functional unit also supports the *spawn* operation (i.e., generate a new task token and issue to the CGRA controller), which is unique in ARENA. If sufficient information is

available ($TASK_{id}$, $TASK_{start}$ and $TASK_{end}$), a new token can be spawn in a single cycle; otherwise, two cycles are required to encode additional information (i.e., $PARAM$, $REMOTE_{start}$, and $REMOTE_{end}$). The $FROM_{node}$ filed will be automatically filled by the CGRA Controller. In Figure 7, there are 4 tiles being able to spawn new tasks (marked in green). The leftmost tiles are connected to two 4-port scratchpad data memory banks. The functional unit can be configured to perform different operations at each cycle based on the control signals from the control memory. The CGRA tiles are granular to support the simultaneous execution of multiple tasks. Specifically, all the tiles are partitioned into 4 groups and a task can be executed by 1, 2, and 4 groups, dynamically managed by the CGRA controller. Note that we choose the simple functional unit design that supports basic operations to minimize the control memory and alleviate the compiler overhead in operation mapping.

Control Memory – The control signals of all the tasks are initially pre-loaded into the control memory. At runtime, tiles iterate over a subset of the control signals to execute specific tasks based on the $TASK_{id}$ in the task token. Each tile requires a 480-byte control memory in our prototype to support all application tasks evaluated in this paper (Section 5). Each task has three execution modes powered by different tile groups. It takes only 8 cycles for the CGRA controller to reconfigure specific tile groups by using the data network to forward the $TASK_{id}$ systolically through the array from right to left.

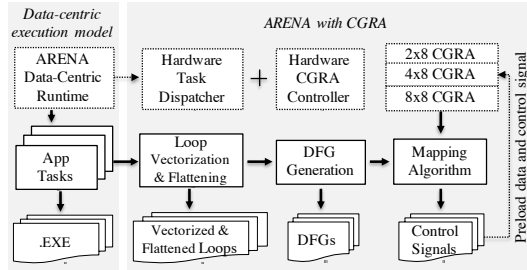


Fig. 8: ARENA data-centric execution model development procedure with compiler toolchain.

CGRA Controller – The CGRA Controller can launch a task (using the task token at the head of `WaitQueue`) to be executed by different groups of the CGRA tiles. Based on the current CGRA utilization status and the data requirement of the target task, the CGRA controller allocate an appropriate number of groups for a waiting task. For example, if the data range required by the target task is less than a quarter of the local data range (i.e., $TASK_{end} - TASK_{start} < (LOCAL_{end} - LOCAL_{start})/4$), only one available group (i.e., 2x8 CGRA tiles) will be allocated to the task. When the target task works on more than half of the local data range (i.e., $TASK_{end} - TASK_{start} > (LOCAL_{end} - LOCAL_{start})/2$), the CGRA controller attempts to allocate all the four groups (i.e., entire 8x8 CGRA tiles) when available (otherwise, two groups are allocated). In addition, there are four queues in the controller to temporarily hold the spawned task tokens, which would be coalesced by the `Coalescing Unit` if any two of them imply continuous task data range and share the same $TASK_{id}$ and $PARAM$. When there are insufficient slots in the queues, the CGRA controller stops fetching tokens from the `WaitQueue` in the Task Dispatcher. Deadlock can

be avoided by providing a memory attached to the CGRA controller for storing the over-spawned task tokens.

Compiler Toolchain for CGRA – Figure 8 shows the development procedure for ARENA using CGRA cluster as the backend. As already mentioned, the CGRA cluster is just one design choice; the ARENA execution model can be realized on alternative back-ends in case the HAF APIs (Table 1) are realized. For example, on a CPU cluster backend, we can adopt MPI non-blocking-send primitives [31] to realize HAF APIs. We use this as one of our baselines in the evaluation. Here, to support the CGRA-cluster backend, an LLVM [45] based design automation toolchain is developed. In particular, a kernel that typically includes a multi-level nested loop is described as a task. As shown in Figure 8, to synthesize an appropriate mapping for a task on the allocated CGRA tiles, the nested loop is first vectorized with a factor that can fully leverage relatively larger CGRA tiles (e.g., 8x8 tiles) in the vectorization pass. Then, the remaining loops are flattened, generating the Control-Data Flow Graph (CDFG) representation, which is an extension of DFG with control dependence edges. We implemented a heuristic method [42] to map the CDFG on various combinations of the tiles (i.e., 2x8, 4x8, and 8x8 tiles) and produce their control signals.

5 EVALUATION

5.1 Environment Setup

The ARENA runtime is evaluated on traditional CPU HPC clusters and our proposed CGRA-based ARENA cluster. We extend the *Structural Simulation Toolkit* (SST) [66] to model a multi-node cluster based on MPI. We model the network topology and package transmit switch in SST using the MACRELS Analytic Model [75]. The token transmit network is modeled as 1D Torus Ring. We implement the task dispatcher, CGRA, and CGRA controller in PyMTL [38], [69], which reports cycle-accurate simulation results for a single node. To obtain the cycle-level simulation result, we implement the dispatcher interface in SST to handle the task tokens. We feed the single-node execution cycles to SST for the performance evaluation. Finally, We use the generated synthesizable Verilog from PyMTL models for single-node power, area, and timing analysis. The detailed simulation and implementation parameters are listed in Table 2.

Technology	45nm
Network Interface	80 Gb/s
Network Topology	1D Torus Ring
Network Switch	1 per node, 1us hop latency
Dispatcher	Filter logic, 8-entry receive queue, 8-entry wait queue, 8-entry send queue
CPU (baseline)	2.6GHz, 20MB 3-level Cache, Out-of-order, x86
CGRA	8 × 8 CGRA, 480-byte control memory per tile, 2-bank 4-port 32KB scratchpad data memory,
CGRA Controller	4 × 4-entry queue for spawned task tokens, Coalescing unit

TABLE 2: RTL and simulation parameters for ARENA.

Applications – We evaluate ARENA using representative HPC and data-analytics workloads: The single-source shortest paths (SSSP) problem (see Section 3.1) is a key subroutine in many data-intensive graph computations. General Matrix Multiply (GEMM) is the core function of linear algebra and deep learning workloads. We assume the matrices are distributed among nodes. Sparse-matrix-vector

multiplication (SPMV) is the fundamental kernel in many scientific & data applications. Here, the distributed matrix is in the Compressed Sparse Row (CSR) format. DNA sequence alignment (DNA) [40] leverages Needleman-Wunsch (NW) algorithm to search the best-matched protein sequences with respect to the target pattern. A Graph convolutional network GCN [29] inference application on the Cora dataset is also evaluated, representing emerging irregular machine learning workloads. We assume the adjacency and feature matrices are distributed among nodes. Finally, an **N-body** simulation application [33] for simulating dynamical particle system is demonstrated, representing traditional scientific simulation workloads. Again, the particle information (e.g., accelerations, velocity, position, collision, etc) are distributively stored and required to be updated per iteration at runtime.

The conventional compute-centric parallel implementations of all the evaluated applications are developed based on state-of-the-art algorithms or derived from the widely-used benchmark suites [22], [63]. For example, the SSSP application is implemented based on [19]. The GCN model is extracted from PyTorch Geometric [29]. The DNA application leverages the NW algorithm from Rodinia [22]. Regarding the ARENA implementation, all the applications are programmed following the ARENA programming models for data-centric asynchronous execution with runtime hardware specialization. Note that GCN and NBody contain numeric distinct functional tasks.

5.2 Evaluation Results

We show the benefits of the data-centric programming model, CGRA hardware acceleration, and the entire ARENA system.

Programming Model – We first show the performance effectiveness of ARENA’s data-centric programming model. Figure 9 illustrates the normalized speedups for conventional compute-centric and ARENA’s models (Both are software implemented based on MPI) with respect to a serial implementation on a single CPU node (i.e., baseline). As can be seen, ARENA’s data-centric execution model shows higher speedups and better scalability in general. This is mainly due to the elimination of synchronization, and the minimization of data communication. On average, ARENA’s data-centric model outperforms the compute-centric counterpart by $1.61\times$ (i.e., $7.82/4.87$) in a 16-node cluster. Specifically, the kernels with higher data parallelism (e.g. SSSP, GEMM, and SPMV) can gain better scalability for both models; for kernels with limited data parallelism such as DNA, the compute-centric model exhibits lower scalability due to massive data dependency and costly remote communication. In this condition, ARENA achieves better scalability by streaming task tokens over the nodes to minimize data movement. Note that the ring network topology limits the scalability when the number of spawned tokens dramatically increases (e.g., SSSP), which implies longer time for a token to wait inside the dispatcher and to arrive at the destination node. Complex datasets may also impact the results of data dependent applications running on ARENA. For example, the SSSP performs better when the input graph has average lower out-degree or higher out-degree with connections to the nodes allocated in contiguous memory locations. This is because each edge in the graph of the SSSP potentially indicates a spawned task in ARENA, which might saturate the task

dispatcher and the ring network when all the nodes burst out spawned tasks for a graph with average higher out-degree. The coalescing unit can alleviate this overhead by coalescing tokens with continuous task data range.

Figure 11 illustrates the normalized data movement breakdown for ARENA’s data-centric model with respect to the compute-centric model for all applications in a 4-node cluster. Compared with the compute-centric HPC cluster, ARENA can eliminate on average 53.9% data movement without any prior knowledge about the data distribution, leading to substantial improvement in energy efficiency. We also observe different data movement patterns across applications. For example, SSSP posts considerable task movement, as it spawns massive fine-grained tasks with discrete data-ranges, which are hard to coalesce. Regarding DNA, the compute-centric implementation is based on OpenMP where all threads are sharing the same copy in global memory [22]. The sub-blocks workload distribution to threads following a zig-zag manner incurs frequent data movement. In ARENA, the data dependency only exists on the edge of the sub-block, which can be explicitly labeled by the parent tasks using the ARENA User-APIs (i.e., $REMOTE_{start}$ and $REMOTE_{end}$ in `ARENA_task_spawn()`), therefore minimizing data movement. Finally, GEMM and NBody comprise coarse-grained tasks and the task-flows require data streaming among the nodes, leading to little task movement or essential data movement as shown in the figure.

CGRA Speedup – Figure 12 shows the normalized speedup of the evaluated applications running on different configurations or combinations of ARENA’s CGRA tile groups (each group is a 2×8 CGRA) with respect to the single node CPU baseline. In general, a larger CGRA tile configuration leads to higher speedups. For DNA, however, the loop-carried data dependency limits the data parallelism, as well as the obtainable speedups ($1.7\times$ speedup at most). On average, $1.3\times$, $2.4\times$, and $3.5\times$ speedups are achieved by ARENA’s 2×8 , 4×8 , and 8×8 CGRA across all the applications and kernels. The 2×8 CGRA exhibits the optimal area-efficiency, showcasing the advantages of runtime hardware specialization.

Overall System – The normalized speedup of ARENA is shown in Figure 10. As can be seen, ARENA maximizes the overall performance by leveraging a data-centric execution model and CGRA in a synergistic and integrated fashion. Instead of fixing the CGRA configuration for each workload, ARENA dynamically allocates and configures the CGRA tiles specifically for a particular task based on the task-carried specification (obtained based on data requirement of the task), as well as the current CGRA resource availability. On average, the compute-centric execution model using the entire CGRAs for each kernel obtains $10.06\times$ speedup on a 16-node cluster, whereas ARENA achieves $21.29\times$ speedup. In other words, ARENA is $2.17\times$ better than the compute-centric with CGRA support). This implies ARENA can leverage the CGRAs in a more efficient way. Compared with Figure 9, we can see that ARENA with CGRAs also gains better scalability (from $1.61\times$ to $2.17\times$ on a 16-node cluster). The performance of DNA does not improve much due to limited acceleration from CGRA. Finally, the compute-centric execution of GEMM does not scale well because synchronization over a larger amount of data creates serious performance bottlenecks.

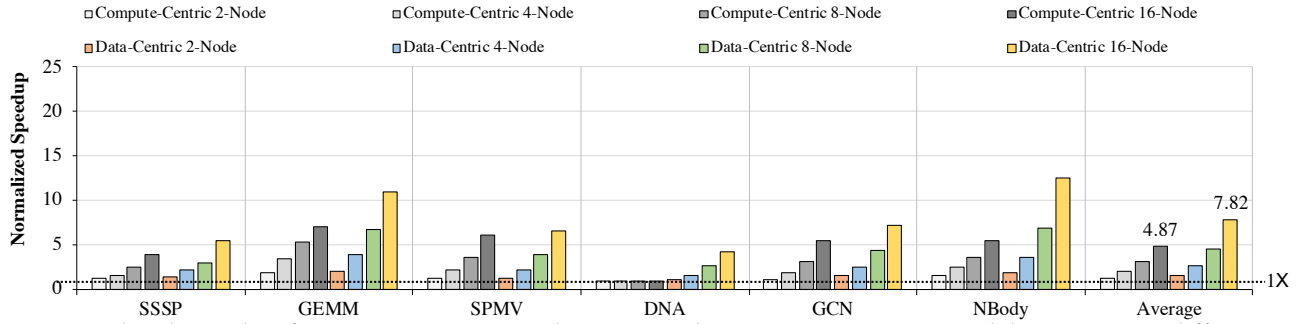


Fig. 9: Normalized speedup for compute-centric and ARENA's data-centric execution models running on different multi-CPU cluster w.r.t. a serial implementation on a single node.

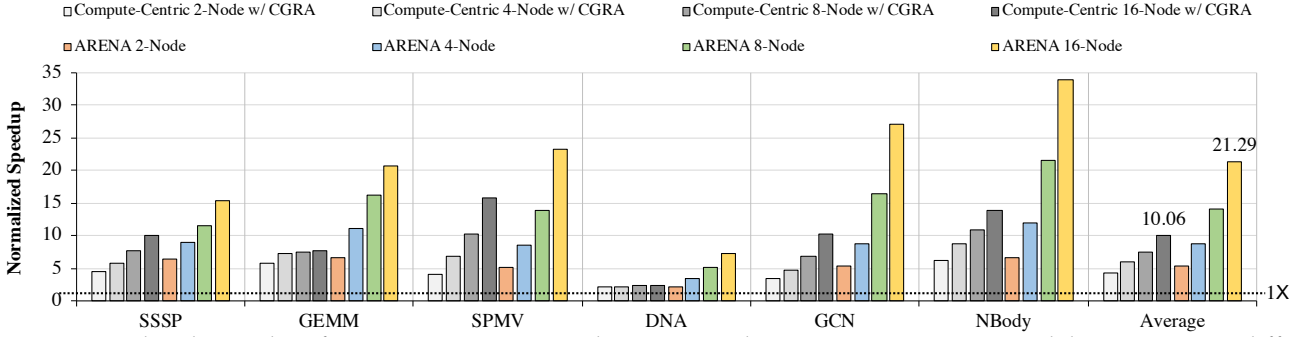


Fig. 10: Normalized speedup for compute-centric and ARENA's data-centric execution models running on different multi-CGRAs cluster w.r.t. a serial implementation on a single node without any acceleration.

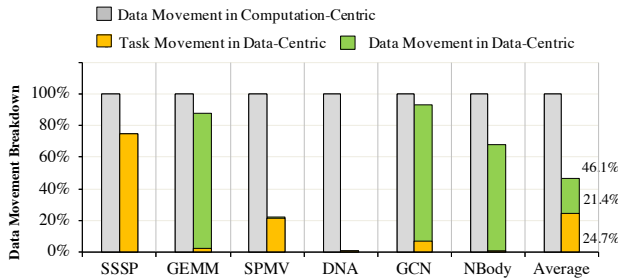


Fig. 11: Normalized data movement breakdown in data-centric model w.r.t. the compute-centric model.

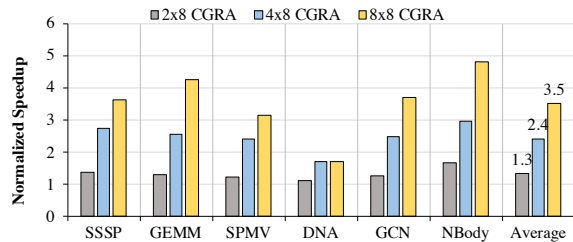


Fig. 12: Normalized CGRA speedup w.r.t. the single node baseline CPU execution without any acceleration.

5.3 RTL Timing, Area and Power

We evaluate the timing, area, and power consumption of ARENA's CGRA-cluster (e.g., CGRAs, CGRA controllers and task dispatchers) using the synthesized Verilog HDL code from PyMTL. We use Synopsys Design Compiler, Cadence Innovus, and Synopsys PrimeTime PX in order to synthesize, place, route, and estimate the power consumption of the designs. We use FreePDK45 with the Nangate standard cell library. Figure 13 illustrates the obtained chip layout. The area and power of the 32KB scratchpad data memory are estimated based on CACTI-6.5. The chip area is 2.19mm × 1.24mm and the operating frequency is 800MHz @ 45nm with on average 759.8mW power consumption.

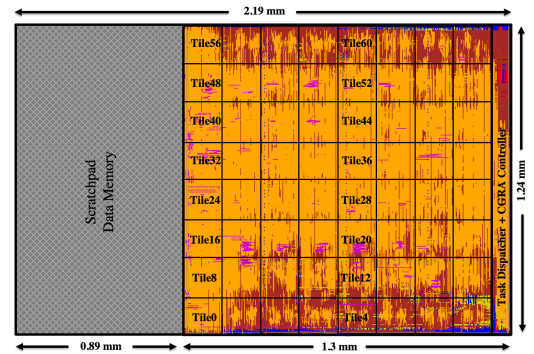


Fig. 13: Chip layout of a single ARENA node.

5.4 Discussion

ARENA leverages CGRA since it provides more flexibility and coarser granularity compared with ASIC and FPGA, respectively. Specifically, it is not realistic to design ASIC accelerators for all seen and unseen kernels running on HPC. On the other hand, the reconfigurability provided by the CGRA perfectly fits the trend of the evolvement of the HPC applications. Meanwhile, CGRA allows specialization even during system execution, which eases the system deployment with lower hardware overhead. Compared with the long reconfiguration time on FPGA, the lower overhead of CGRA reconfiguration allows multiple users to share the accelerator dynamically and run applications with divergent characteristics.

ARENA's programming/execution model can be applied on different devices as long as the hardware abstract functions (listed in Table 1) are supported. Groute [13] is a data-centric asynchronous runtime environment built on multi-GPU cluster for processing graphs. However, pure software-based implementation on GPUs cannot benefit from ARENA's per-task hardware specialization and incurs excessive overhead for task spawn, dispatch, and execution.

CGRA-based architecture is a promising alternative to

GPUs for throughput-oriented applications, particularly scientific machine learning, in HPC. We compare the CGRA-based SambaNova SR10-8 Reconfigurable Dataflow Unit (RDU) [28], [64] with the NVIDIA P100 GPU using batched matrix multiplication kernels in different sizes (as shown in Table 3). The results show that the SambaNova RDU outperforms the P100 GPU in terms of end-to-end latency for large kernel size (e.g., 2048x2048). In addition, the RDU achieves 715 $\times$ improvement over GPU in terms of throughput thanks to its data-flow type of execution. This comes at the cost of increased end-to-end latency for smaller matrices (e.g., 29% extra latency for 1024x1024 GEMM). In contrast, ARENA does not sacrifice the performance in latency. We conservatively (regardless of the technology) estimate the area-efficiency (i.e., performance/area) for GEMM (1024x1024) to provide a fair comparison, which shows that ARENA achieves 3.88 $\times$ and 2.4 $\times$ improvement over SambaNova RDU and GPU, respectively. To be specifically, ARENA improves the performance of 1024x1024 GEMM by 3.5 $\times$ over the CPU baseline with 2.7mm² area overhead at 45nm, while the P100 GPU achieves 329 $\times$ speedup with 610mm² area at 16nm and the SambaNova chip takes 708mm² with 236 $\times$ end-to-end performance improvement.

	Sambanova	GPU-P100	ARENA
Tech (nm)	7	16	45
Area (mm ²)	708	610	2.7
Latency (ms) 256x256	0.1716	0.0164	4.296
Latency (ms) 512x512	0.2524	0.0571	15.49
Latency (ms) 1024x1024	0.4987	0.358	33.68
Latency (ms) 2048x2048	1.2424	2.314	177.2
Area-Efficiency 1024x1024 1/sec/mm ²	2.832	4.58	10.997

TABLE 3: Performance (end-to-end latency) and area-efficiency (performance/area) evaluation of GEMM on SambaNova, GPU, and ARENA CGRA.

6 RELATED WORK

We summarize related work regarding the ring network, clusters of reconfigurable architecture, and the task-based execution model in a ring network.

Ring Network. The ring network has been adopted in real multi-processors design [11] (e.g., Intel Xeon-Phi [23]) and logically studied for efficient collective communication [8], [10], [47], [48], [49], [77]. ARENA's ring network is a logical topology that is independent from and set up on top of the underneath physical network. The essential data is still transferred in an all-to-all fashion while only the task tokens are delivered on the ring to keep the consistency of ordering between the task spawn and task execution, which simplifies the programming model. Here we use the ring network for task token delivery because: (I) the ring network provides simpler routing mechanism [11] or even constructing logically routerless network [8], [48], [49], [77]; (II) It has been shown that by partitioning data into small chunks and transmitting them along a ring network in a pipeline fashion, the ring algorithm can offer near optimal bandwidth for most standard collective communication [57], which is the similar case here; (III) As practiced by NCCL [56], the ring network can be easily constructed upon a variety

of network topologies, including the hypercube network in DGX-1 [57], the crossbar network in DGX-2 [58], and the hierarchical tree network of PCI-e [47]. Particularly, the V100-DGX-1 even constructed a backbone ring for fast interconnect communication, which improves the collective communication bandwidth by 1.6 $\times$ and 2.4 $\times$ on 4 GPUs and 8 GPUs, respectively [47].

On the other hand, the ring network has been criticized for easy saturation due to linear increased latency with more node [7], [37], [49]. In ARENA, we avoid such a saturation problem through: (a) a routerless task execution model among nodes; and (b) the dynamically task allocation and dispatching mechanism.

Reconfigurable Hardware Cluster. Clusters incorporating reconfigurable devices such as FPGAs [26], [30], [44], [65], [70] and CGRAs [39], [64], [71] have already been showcased by existing works [28], [50], [72]. On one hand, Putnam et al. from Microsoft propose the reconfigurable Catapult fabric [65], where each instantiation consists of a 6x8 2D torus of Xilinx Stratix-V FPGAs. Every FPGA is connected to a CPU server via PCI-e and directly links to other FPGAs through the SAS cables. This 1632-node FPGA cluster has been adopted for document ranking of the Bing search engine. Nearly 95% performance increase has been demonstrated with only a 10% extra power budget. However, each FPGA in Catapult is specialized to a single application kernel during runtime. Reconfiguration takes several milliseconds. Even reloading models without changing the computation logic can take up to 250 microseconds. On the other hand, the data-centric execution model allowing different tasks working on the same set of data in an HPC node requires rapid dynamic reconfiguration. Gazzano et al. propose R-Grid, a complete grid infrastructure for distributed high-performance computing using dynamically reconfigurable FPGAs [26]. Knodel et al. virtualized the FPGA resources and propose adapted service models in a cloud context [44]. Tarafdar et al. discuss how FPGAs of a cloud data center can be flexibly connected based on a logical kernel and a mapping file [70]. Zhang et al. adopt a cluster of six Xilinx VC709 FPGA for cooperative convolutional neural network inference [79]. Regarding to the CGRAs, the SambaNova DataScale system incorporates 8 reconfigurable-dataflow-units (RDUs) derived from Plasticine [64], claiming higher performance than a thousand GPUs for training extremely large deep-learning models. PPA [61] exploits pipeline parallelism in streaming applications to create a CGRA-like pipeline to execute streaming media applications. Samsung proposes to adopt the CGRA cluster for medical volume image rendering [39]. HammerBlade [71] aims at designing a rack-scale cluster for ML and Graphs. Their ASIC is composed of general-purpose cores and specialized CGRAs.

Asynchronous Task Execution. The asynchronous tasks execution has been studied in many graph processing frameworks [13], [54], [55]. They propose software-level data-centric approaches to implement irregular graph kernels on multi-node clusters [55] and GPUs platform [54]. The work that is most relevant to ARENA is Groute [13], which is an asynchronous runtime environment for processing irregular graphs. In Groute, the GPUs form a logical ring network with each GPU conserves a worklist. Tasks are encapsulated as messages passing along the ring. ARENA is motivated by

Groute. However, Groute is a pure software implementation based on general-purpose GPUs, thus cannot benefit from hardware specialization. Additionally, for Groute, the routing policy, which specifies the action when input is received, as well as memory consistency and ownership, are all defined and maintained by the users, which can be complicated, tedious and error-prone (e.g., imagine if data dependency occurs at runtime, users have to manually locate them and fetch in corresponding tasks). In ARENA, these are designed and supported by hardware. Furthermore, global coordination and work counting in Groute are centralized and managed by the CPU whereas in ARENA, all of them are distributedly processed.

7 CONCLUSION

In this paper, we propose an asynchronous-reconfigurable-accelerator-ring architecture for next generation data-driven high-performance computing. Through the co-design of architecture and programming model, ARENA is able to bring computation tasks in the form of CGRA-specialized hardware accelerators to the data, rather than the reverse as in contemporary compute-centric and dataflow architectures, improving performance and reducing data-movement.

ACKNOWLEDGEMENT

This work was mainly supported by the Compute-Flow-Architecture (CFA) project under PNNL's Data-Model-Convergence Initiative. It was also partially supported by the SO(DA)² and FALLACY projects under DMC. The evaluation platforms were supported by the U.S. DOE Office of Science, Office of Advanced Scientific Computing Research, under award 66150: "CENATE - Center for Advanced Architecture Evaluation". The Pacific Northwest National Laboratory is operated by Battelle for the U.S. Department of Energy under contract DE-AC05-76RL01830.

REFERENCES

- [1] NVlink power. Accessed April 10, 2020. <https://rb.gy/ew3vf2>.
- [2] Paderborn University Will Offer Intel CPU-FPGA Cluster for Researchers. <https://rb.gy/2vc51j>, 2017.
- [3] DOE Joint Genome Institute. <http://jgi.doe.gov>, 2018.
- [4] GridLAB-D. <http://www.gridlabd.org>, 2018.
- [5] Exascale Combustion Co-Design Center (ExaCT). <https://crd.lbl.gov/projects/combustion-codesign>, 2020.
- [6] NWChem: Open Source High-Performance Computational Chemistry. <http://www.nwchem-sw.org>, 2020.
- [7] T. W. Ainsworth and T. M. Pinkston. On characterizing performance of the cell broadband engine element interconnect bus. In *First International Symposium on Networks-on-Chip (NOCS'07)*, pages 18–29. IEEE, 2007.
- [8] F. Alazemi, A. Azizimazreah, B. Bose, and L. Chen. Routerless network-on-chip. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018.
- [9] P. Alvaro, T. Condie, N. Conway, K. Elmeleegy, J. M. Hellerstein, and R. C. Sears. BOOM: Data-centric programming in the datacenter. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2009-113*, 2009.
- [10] A. A. Awan, K. Hamidouche, A. Venkatesh, and D. K. Panda. Efficient large message broadcast using NCCL and CUDA-aware MPI for deep learning. In *Proceedings of the 23rd European MPI Users' Group Meeting*, pages 15–22, 2016.
- [11] L. A. Barroso and M. Dubois. The performance of cache-coherent ring-based multiprocessors. In *Proceedings of the 20th annual international symposium on computer architecture*, pages 268–277, 1993.
- [12] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken. Legion: Expressing locality and independence with logical regions. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–11. IEEE, 2012.
- [13] T. Ben-Nun, M. Sutton, S. Pai, and K. Pingali. Groute: An Asynchronous Multi-GPU Programming Model for Irregular Computations. *ACM SIGPLAN Notices*, 52(8):235–248, 2017.
- [14] R. H. Bisseling and W. F. McColl. Scientific computing on bulk synchronous parallel architectures. 1993.
- [15] R. D. Blumofe, C. F. Joerg, B. C. Kuszmaul, C. E. Leiserson, K. H. Randall, and Y. Zhou. Cilk: An efficient multithreaded runtime system. *Journal of parallel and distributed computing*, 1996.
- [16] D. Bonachea and P. Hargrove. GASNet Specification, v1. 8.1. 2017.
- [17] D. Bonachea and P. H. Hargrove. GASNet-EX: A high-performance, portable communication library for exascale. In *International Workshop on Languages and Compilers for Parallel Computing*, pages 138–158. Springer, 2018.
- [18] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungrun, E. Shiu, R. Thakur, D. Kim, A. Kuusela, A. Knies, P. Ranganathan, and O. Mutlu. Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 316–331, 2018.
- [19] A. Buluç and K. Madduri. Parallel breadth-first search on distributed memory systems. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2011.
- [20] B. L. Chamberlain, D. Callahan, and H. P. Zima. Parallel programmability and the chapel language. *The International Journal of High Performance Computing Applications*, 21(3):291–312, 2007.
- [21] P. Charles, C. Grothoff, V. Saraswat, C. Donawa, A. Kielstra, K. Ebcioğlu, C. Von Praun, and V. Sarkar. X10: an object-oriented approach to non-uniform cluster computing. *Acm Sigplan Notices*, 40(10):519–538, 2005.
- [22] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE international symposium on workload characterization (IISWC)*, pages 44–54. Ieee, 2009.
- [23] G. Chrysos. Intel® xeon phi coprocessor (codename knights corner). In *2012 IEEE Hot Chips 24 Symposium (HCS)*, pages 1–31. IEEE, 2012.
- [24] B. De Sutter, P. Raghavan, and A. Lambrechts. Coarse-grained reconfigurable array architectures. In *Handbook of signal processing systems*, pages 427–472. Springer, 2019.
- [25] J. Dean and S. Ghemawat. MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.
- [26] J. Dondo Gazzano, F. Sanchez Molina, F. Rincon, and J. C. López. Integrating reconfigurable hardware-based grid for high performance computing. *The Scientific World Journal*, 2015, 2015.
- [27] J. Dongarra, L. Grigori, and N. J. Higham. Numerical algorithms for high-performance computational science. *Philosophical Transactions of the Royal Society A*, 378(2166):20190066, 2020.
- [28] M. Emani, V. Vishwanath, C. Adams, M. E. Papka, R. Stevens, L. Florescu, S. Jairath, W. Liu, T. Nama, and A. Sajeeth. Accelerating Scientific Applications With SambaNova Reconfigurable Dataflow Architecture. *Computing in Science & Engineering*, 2021.
- [29] M. Fey and J. E. Lenssen. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- [30] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, et al. A configurable cloud-scale DNN processor for real-time AI. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–14. IEEE, 2018.
- [31] E. Gabriel, G. E. Fagg, G. Bosilca, T. Angskun, J. J. Dongarra, J. M. Squyres, V. Sahay, P. Kambadur, B. Barrett, A. Lumsdaine, R. H. Castain, D. J. Daniel, R. L. Graham, and T. S. Woodall. Open MPI: Goals, concept, and design of a next generation MPI implementation. In D. Kranzlmüller, P. Kacsuk, and J. Dongarra, editors, *European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting*, pages 97–104. Springer, 2004.
- [32] T. Geng, T. Wang, A. Sanaullah, C. Yang, R. Xu, R. Patel, and M. Herboldt. FPDeep: Acceleration and load balancing of CNN training on FPGA clusters. In *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2018.
- [33] S. Green. Particle simulation using cuda. *NVIDIA whitepaper*, 6:121–128, 2010.
- [34] M. Hamzeh, A. Shrivastava, and S. Vrudhula. Branch-aware loop mapping on CGRAs. In *Proceedings of the 51st Annual Design Automation Conference*, pages 1–6, 2014.
- [35] K. Hazelwood, S. Bird, D. Brooks, S. Chintala, U. Diril, D. Dzhulgakov, M. Fawzy, B. Jia, Y. Jia, A. Kalro, J. Law, K. Lee, J. Lu, P. Noordhuis, M. Smelyanskiy, L. Xiong, and X. Wang. Applied ma-

- chine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 620–629. IEEE, 2018.
- [36] M. Horowitz. 1.1 computing’s energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14. IEEE, 2014.
- [37] N. E. Jerger and L.-S. Peh. On-chip networks. *Synthesis Lectures on Computer Architecture*, 4(1):1–141, 2009.
- [38] S. Jiang, P. Pan, Y. Ou, and C. Batten. PyMTL3: a Python framework for open-source hardware modeling, generation, simulation, and verification. *IEEE Micro*, 40(4):58–66, 2020.
- [39] S. Jin, S. Lee, M.-K. Chung, Y. Cho, and S. Ryu. Implementation of a volume rendering on coarse-grained reconfigurable multiprocessor. In *International Conference on Field-Programmable Technology*, 2012.
- [40] M. Johnson, I. Zaretskaya, Y. Raytselis, Y. Merezuk, S. McGinnis, and T. L. Madden. NCBI BLAST: a better web interface. volume 36, pages W5–W9, 04 2008.
- [41] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, et al. In-datacenter performance analysis of a tensor processing unit. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.
- [42] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh. Hycube: A cgra with reconfigurable single-cycle multi-hop interconnect. In *Proceedings of the 54th Annual Design Automation Conference 2017*, pages 1–6, 2017.
- [43] G. Kestor, R. Gioiosa, D. J. Kerbyson, and A. Hoisie. Quantifying the energy cost of data movement in scientific applications. In *2013 IEEE international symposium on workload characterization (IISWC)*, pages 56–65. IEEE, 2013.
- [44] O. Knodel, P. Lehmann, and R. G. Spallek. RC3E: Reconfigurable accelerators in data centres and their provision by adapted service models. In *2016 IEEE 9th International Conference on Cloud Computing (CLOUD)*, pages 19–26. IEEE, 2016.
- [45] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization*, 2004.
- [46] K. Lee. Introducing big basin: Our next-generation ai hardware, 2017.
- [47] A. Li, S. L. Song, J. Chen, J. Li, X. Liu, N. R. Tallent, and K. J. Barker. Evaluating Modern GPU Interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE Transactions on Parallel and Distributed Systems*, 31(1):94–110, 2019.
- [48] T.-R. Lin, D. Penney, M. Pedram, and L. Chen. A Deep Reinforcement Learning Framework for Architectural Exploration: A Routerless NoC Case Study. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018.
- [49] S. Liu, T. Chen, L. Li, X. Feng, Z. Xu, H. Chen, F. Chong, and Y. Chen. IMR: High-performance low-cost multi-ring NoCs. *IEEE Transactions on Parallel and Distributed Systems*, 2015.
- [50] J. C. Lyke, C. G. Christodoulou, G. A. Vera, and A. H. Edwards. An introduction to reconfigurable systems. *Proceedings of the IEEE*, 103(3):291–317, 2015.
- [51] P. J. Maechling. Exascale Applications in Seismic Hazard Analysis. http://www.nics.tennessee.edu/files/pdf/Maechling_Exascale_072910_v9.pdf, 2018.
- [52] D. Molka, D. Hackenberg, R. Schöne, and M. S. Müller. Characterizing the energy consumption of data transfers and arithmetic operations on x86-64 processors. In *International conference on green computing*, pages 123–133. IEEE, 2010.
- [53] G. E. Moore. Cramming more components onto integrated circuits, 1965.
- [54] R. Nasre, M. Burtcher, and K. Pingali. Data-driven versus topology-driven irregular computations on GPUs. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pages 463–474. IEEE, 2013.
- [55] D. Nguyen, A. Lenharth, and K. Pingali. A lightweight infrastructure for graph analytics. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 456–471, 2013.
- [56] NVIDIA. NVIDIA Collective Communications Library (NCCL-V2). <https://developer.nvidia.com/nccl>.
- [57] NVIDIA. NVIDIA DGX-1 System Architecture White Paper, 2017.
- [58] NVIDIA. NVIDIA DGX-2H The World’s Most Powerful System for The Most Complex AI Challenges, 2018.
- [59] I. Ohmura, G. Morimoto, Y. Ohno, A. Hasegawa, and M. Taiji. MDGRAPE-4: a special-purpose computer system for molecular dynamics simulations. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 2014.
- [60] S. Páll, M. J. Abraham, C. Kutzner, B. Hess, and E. Lindahl. Tackling Exascale Software Challenges in Molecular Dynamics Simulations with GROMACS. In S. Markidis and E. Laure, editors, *Solving Software Challenges for Exascale*, pages 3–27, Cham, 2015. Springer International Publishing.
- [61] H. Park, Y. Park, and S. Mahlke. Polymorphic pipeline array: a flexible multicore accelerator with virtualized execution for mobile multimedia applications. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 370–380, 2009.
- [62] J. M. Perez, R. M. Badia, and J. Labarta. Handling task dependencies under strided and aliased references. In *Proceedings of the 24th ACM International Conference on Supercomputing*, pages 263–274, 2010.
- [63] L.-N. Pouchet and S. Grauer-Gray. Polybench: The polyhedral benchmark suite. URL: <http://www.cs.ucla.edu/pouchet/software/polybench>, 2012.
- [64] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun. Plasticine: A reconfigurable architecture for parallel patterns. In *ACM/IEEE 44th Annual International Symposium on Computer Architecture*, 2017.
- [65] A. Putnam, A. M. Caulfield, et al. A reconfigurable fabric for accelerating large-scale datacenter services. In *ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, 2014.
- [66] A. F. Rodrigues, G. R. Voskuilen, S. D. Hammond, and K. S. Hemmert. Structural Simulation Toolkit (SST). Technical report, Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), 2016.
- [67] J. Shalf. The future of computing beyond Moore’s law. *Philosophical Transactions of the Royal Society A*, 378(2166):20190061, 2020.
- [68] D. E. Shaw, J. Grossman, J. A. Bank, B. Batson, J. A. Butts, J. C. Chao, M. M. Deneroff, R. O. Dror, A. Even, C. H. Fenton, et al. Anton 2: raising the bar for performance and programmability in a special-purpose molecular dynamics supercomputer. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014.
- [69] C. Tan, C. Xie, A. Li, K. J. Barker, and A. Tumeo. OpenCGRA: An Open-Source Unified Framework for Modeling, Testing, and Evaluating CGRAs. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*, pages 381–388. IEEE, 2020.
- [70] N. Tarafdar, T. Lin, E. Fukuda, H. Bannazadeh, A. Leon-Garcia, and P. Chow. Enabling flexible network FPGA clusters in a heterogeneous cloud data center. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 237–246, 2017.
- [71] M. B. Taylor, A. Sampson, C. Batten, Z. Zhang, L. Ceze, M. Oskin, and D. Richmond. The HammerBlade: An ML-Optimized Supercomputer for ML and Graphs. <https://sampl.cs.washington.edu/tvmconf/slides/Michael-Taylor-HammerBlade.pdf>, 2020.
- [72] R. Tessier, K. Pocek, and A. DeHon. Reconfigurable computing architectures. *Proceedings of the IEEE*, 103(3):332–354, 2015.
- [73] D. Trebotich, M. F. Adams, S. Molins, C. I. Steefel, and C. Shen. High-Resolution Simulation of Pore-Scale Reactive Transport Processes Associated with Carbon Sequestration. *Computing in Science Engineering*, 16(6):22–31, Nov 2014.
- [74] L. G. Valiant. A bridging model for parallel computation. *Communications of the ACM*, 33(8):103–111, 1990.
- [75] F. Versolatto and A. Tonello. An MTL Theory Approach for the Simulation of MIMO Power-Line Communication Channels. *Power Delivery, IEEE Transactions on*, 26:1710–1717, 07 2011.
- [76] J. S. Vetter, R. Brightwell, et al. Extreme heterogeneity 2018-productive computational science in the era of extreme heterogeneity: Report for DOE ASCR workshop on extreme heterogeneity. Technical report, USDOE Office of Science (SC), Washington, DC (United States), 2018.
- [77] L. Wang, L. Liu, J. Han, X. Wang, S. Yin, and S. Wei. Achieving Flexible Global Reconfiguration in NoCs using Reconfigurable Rings. *IEEE Transactions on Parallel and Distributed Systems*, 2019.
- [78] L.-W. Wang. Divide-and-conquer quantum mechanical material simulations with exascale supercomputers. *National Science Review*, 1(4):604–617, 2014.
- [79] C. Zhang, D. Wu, J. Sun, G. Sun, G. Luo, and J. Cong. Energy-efficient CNN implementation on a deeply pipelined FPGA cluster. In *Proceedings of the 2016 International Symposium on Low Power Electronics and Design*, pages 326–331, 2016.